

OBSERVACIONES DE LA PRÁCTICA

Juan Sebastián Rojas Marín, 202610140, js.rojasm123@uniandes.edu.co

Katy Teran Mercado, 202614247, k.teranm@uniandes.edu.co

Juan José Ricon Santos, 202615789, jj.rincons1@uniandes.edu.co

Ambientes de pruebas

Computador 1	
Procesador	12th Gen Intel(R) Core (TM) i5-1235U (1.30 GHz)
Memoria RAM (GB)	16
Sistema Operativo	Windows 11

Tabla 1. Especificaciones del computador para ejecutar las pruebas de rendimiento.

Para realizar las siguientes pruebas, es importante que las ejecuten utilizando el archivo de datos **large**, ya que es con este conjunto donde realmente se puede observar el comportamiento y la eficiencia de los algoritmos en contextos más exigentes.

Ordenamientos Iterativos

Resultados para ordenamientos iterativos con Array List

Porcentaje de la muestra	Insertion (Array List)	Sort	Selection (Array List)	Sort	Shell (Array List)	Sort
0.50%	0.7		0.63		0.39	
5.00%	46.55		64.91		6.84	
10.00%	169.41		276.61		11.01	
20.00%	686.99		1124.63		23.44	
30.00%	1630.19		2601.32		57.33	
50.00%	4837.04		7765.33		92.05	
80.00%	12986.86		22593.55		207.59	
100.00%	20726.92		40054.33		272.93	

Tabla 2. Resultados en computador 1 para ordenamientos iterativos con Array List.

Resultados para ordenamientos iterativos con Linked List

Porcentaje de la muestra	Insertion (Linked List)	Sort	Selection (Linked List)	Sort	Shell (Linked List)	Sort
0.50%	2.31		11.01		1.58	
5.00%	2451.8		5813.15		242.2	

10.00%	31127.41	32737.73	1176.08
20.00%	NO COMPLETADO	NO COMPLETADO	5160.44
30.00%	NO COMPLETADO	NO COMPLETADO	13126.9
50.00%	NO COMPLETADO	NO COMPLETADO	41175.57
80.00%	NO COMPLETADO	NO COMPLETADO	NO COMPLETADO
100.00%	NO COMPLETADO	NO COMPLETADO	NO COMPLETADO

Tabla 3. Resultados en computador 1 para ordenamientos iterativos con Single Linked List.

Ordenamientos Recursivos

Resultados para ordenamientos recursivos con Array List

Porcentaje de la muestra	Merge (Array List)	Sort Quick (Array List)	Sort
0.50%	0.52	0.21	
5.00%	8.93	3.88	
10.00%	17.24	7.28	
20.00%	64.71	24.69	
30.00%	71.36	51.63	
50.00%	96.77	126.8	
80.00%	172.5	257.57	
100.00%	200.77	405.81	

Tabla 4. Resultados en computador 1 para ordenamientos recursivos con Array List.

Resultados para ordenamientos recursivos con Linked List

Porcentaje de la muestra	Merge (Linked List)	Sort Quick (Linked List)	Sort
0.50%	0.87	0.76	
5.00%	54.73	166.34	
10.00%	171.03	683.33	
20.00%	689.77	6880.31	
30.00%	1547.44	21130.68	
50.00%	4228.3	67454.42	
80.00%	NO COMPLETADO	NO COMPLETADO	
100.00%	NO COMPLETADO	NO COMPLETADO	

Tabla 5. Resultados en computador 1 para ordenamientos recursivos con Single Linked List.

Una vez alla llenado l

Comparación de tiempo mejores algoritmos de ordenamiento

Complete esta sección únicamente cuando se le indique en las instrucciones. En este punto, deberá comparar el tiempo de ejecución entre el mejor algoritmo de ordenamiento recursivo y el mejor algoritmo de ordenamiento iterativo, los cuales debió haber identificado en el punto anterior.

Por favor, registre las mediciones en el espacio correspondiente de la tabla, especificando claramente:

- El nombre de cada algoritmo utilizado
- El tipo de estructura de datos empleada (*ArrayList* o *SingleLinkedList*)

Asegúrese de completar todos los campos solicitados en la tabla e incluir los nombres según corresponda, no es necesario que vuelva a correr las pruebas esta información la puede sacar de las tablas anteriores.

Computador 1

Porcentaje de la muestra	Algoritmo iterativo SHELL_SORT (ARRAYLIST)	Algoritmo recursivo MERGE_SORT (ARRAYLIST)
0.50%	0.39	0.52
5.00%	6.84	8.93
10.00%	11.01	17.24
20.00%	23.44	64.71
30.00%	57.33	71.36
50.00%	92.05	96.77
80.00%	207.59	172.5
100.00%	272.93	200.77

Tabla 6. Resultados en computador 1 mejores algoritmos

Preguntas de análisis parte 1

1. **¿Cómo varía el comportamiento de cada algoritmo con respecto al tamaño de los datos?**

Para responder, agregue líneas de tendencia en las gráficas de las pestañas 04 a 08. Observe si el crecimiento del tiempo de ejecución es lineal, cuadrático o de otro tipo, y relacione ese patrón con la complejidad teórica esperada de cada algoritmo.

RTA:

Al revisar las gráficas vimos que, a medida que aumenta el tamaño de la muestra, también aumenta el tiempo de ejecución de todos los algoritmos, pero no todos crecen al mismo ritmo.

En Insertion Sort y Selection Sort el crecimiento es bastante pronunciado. Por ejemplo, tomando como referencia los resultados de 1.000 a 10.000 datos, Insertion Sort pasa de 169,41 ms a 20.726,92 ms y Selection Sort de 276,61 ms a 40.054,33 ms. La forma de sus curvas se va haciendo cada vez más inclinada, lo que se acerca al comportamiento cuadrático esperado de

estos algoritmos, cuya complejidad es $O(n^2)$.

En Shell Sort también aumenta el tiempo, pero la curva crece de una forma mucho más moderada. Pasa de 11,01 ms con 1.000 datos a 272,93 ms con 10.000. En este caso no hay una única complejidad que se pueda aplicar siempre, porque depende de la secuencia de saltos utilizada, pero en nuestros resultados su crecimiento fue claramente menor que el de Insertion y Selection.

Merge Sort presentó una tendencia bastante estable. Pasó de 17,24 ms con 1.000 datos a 200,77 ms con 10.000. Ese crecimiento más controlado tiene relación con su complejidad teórica de $O(n \log n)$.

Por último, Quick Sort comenzó con tiempos muy bajos, pero su crecimiento fue más fuerte en las muestras grandes. Con 1.000 datos registró 7,28 ms y con 10.000 llegó a 405,81 ms. Aunque en promedio su complejidad también es $O(n \log n)$, en nuestras pruebas su curva creció más que la de Merge Sort.

En general, las gráficas muestran que Insertion y Selection son los que más se ven afectados cuando crece la cantidad de datos, mientras que Shell, Merge y Quick presentan un crecimiento más moderado, aunque con diferencias entre ellos.

2. ¿Qué diferencias observas en el rendimiento de un mismo algoritmo al utilizar ArrayList frente a SingleLinkedList?

Analice esta comparación en las pestañas 04 a 08, donde se muestran ambos casos. Justifique su respuesta con base en las líneas de tendencia.

RTA:

Al comparar un mismo algoritmo en las dos estructuras vimos que ArrayList obtuvo mejores tiempos que SingleLinkedList en todas las muestras que logramos completar. Además, la diferencia entre ambas estructuras se hace mucho mayor cuando aumenta el número de datos.

Por ejemplo, en Shell Sort con 1.000 datos obtuvimos 11,01 ms en ArrayList y 1.176,08 ms en SingleLinkedList. Con 5.000 datos la diferencia aumenta mucho más, con 92,05 ms en ArrayList frente a 41.175,57 ms en SingleLinkedList.

Algo similar ocurre con Merge Sort. Con 1.000 datos registra 17,24 ms en ArrayList y 171,03 ms en SingleLinkedList, mientras que con 5.000 datos pasa a 96,77 ms y 4.228,30 ms respectivamente. En Quick Sort la diferencia es todavía mayor: con 5.000 datos se obtuvieron 126,80 ms en ArrayList y 67.454,42 ms en SingleLinkedList.

Las gráficas también muestran que esta diferencia no es constante. Las líneas comienzan relativamente cercanas con las muestras pequeñas, pero se van separando bastante a medida que aumenta el tamaño de la muestra.

Además, en SingleLinkedList varias ejecuciones las dejamos como “NO COMPLETADO” porque superaron los 10 minutos sin mostrar un tiempo final. Esto ocurrió desde los 2.000 datos para Insertion y Selection y en tamaños mayores para los demás algoritmos.

Con base en esto, consideramos que ArrayList fue mucho más favorable para los ordenamientos realizados. Esto también tiene sentido por la forma en que funciona cada estructura, ya que ArrayList permite acceder directamente a una posición, mientras que en SingleLinkedList hay que recorrer los nodos para llegar hasta determinados elementos.

3. ¿Qué algoritmo iterativo mostró el mejor comportamiento general en términos de tiempo y cantidad de datos en ambas estructuras de datos?

Justifique su elección comparando las gráficas (04 Insertion Sort), (05 Selection Sort) y (06 Shell Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA:

Consideramos que el mejor comportamiento general lo tuvo Shell Sort.

En los resultados se nota una diferencia bastante grande. Con 1.000 datos, Shell Sort registra 11,01 ms, mientras que Insertion Sort tarda 169,41 ms y Selection Sort 276,61 ms. Al llegar a los 10.000 datos, Shell Sort tarda 272,93 ms, frente a 20.726,92 ms de Insertion Sort y 40.054,33 ms de Selection Sort.

Al mirar los resultados obtenidos en la otra estructura, Shell Sort también presentó una ventaja. Con 1.000 datos tardó 1.176,08 ms, mientras que Insertion llegó a 31.127,41 ms y Selection a 32.737,73 ms. Además, Insertion y Selection superaron los 10 minutos desde los 2.000 datos y no mostraron un tiempo final, mientras que Shell Sort logró completar las pruebas hasta los 5.000 datos.

Por eso escogimos Shell Sort como el mejor algoritmo iterativo, ya que tuvo menores tiempos y también consiguió trabajar con muestras de mayor tamaño antes de superar el límite establecido para las pruebas.

4. ¿Qué algoritmo recursivo presentó el mejor desempeño general según los tiempos registrados y la forma de la línea de tendencia?

Justifique su respuesta basándose en las pestañas 07 (Merge Sort) y 08 (Quick Sort) y apoyándose en las gráficas de las pestañas 2 y 3.

RTA:

Al comparar Merge Sort y Quick Sort, vimos que Quick Sort fue más rápido en las muestras pequeñas, pero esa ventaja cambió cuando aumentó la cantidad de datos.

Por ejemplo, con 1.000 datos Quick Sort tarda 7,28 ms frente a 17,24 ms de Merge Sort. Con 2.000 datos también tiene ventaja, con 24,69 ms frente a 64,71 ms, y con 3.000 datos registra 51,63 ms frente a 71,36 ms.

Sin embargo, a partir de los 5.000 datos Merge Sort empieza a presentar mejores resultados. En ese tamaño registra 96,77 ms frente a 126,80 ms de Quick Sort. Con 8.000 datos los tiempos son 172,50 ms y 257,57 ms respectivamente, y con 10.000 datos Merge Sort tarda 200,77 ms mientras Quick Sort llega a 405,81 ms.

La gráfica permite ver este cambio porque Quick Sort comienza con una ventaja, pero su crecimiento se vuelve más pronunciado en las últimas muestras. Merge Sort, en cambio,

mantiene una tendencia más estable.

Esta diferencia también se observa en las pruebas donde ambos pudieron ejecutarse con SingleLinkedList. Por ejemplo, con 5.000 datos Merge Sort tardó 4.228,30 ms y Quick Sort 67.454,42 ms. Las muestras de 8.000 y 10.000 no fueron completadas por ninguno de los dos porque superaron los 10 minutos.

Por estas razones seleccionamos Merge Sort como el mejor algoritmo recursivo. Aunque Quick Sort fue más rápido en las muestras pequeñas, Merge Sort mostró un mejor comportamiento general cuando aumentó la cantidad de datos.

5. Con base en su análisis visual y los datos recolectados, ¿cuáles algoritmos seleccionan como los “mejores” en cada categoría (iterativo y recursivo)?

Indique también la estructura de datos en la que se desempeñó mejor cada uno.

RTA:

Seleccionamos Shell Sort como el mejor algoritmo iterativo y Merge Sort como el mejor algoritmo recursivo.

En el caso de Shell Sort, fue el iterativo que presentó los menores tiempos y logró trabajar con muestras más grandes que Insertion Sort y Selection Sort. Para los recursivos escogimos Merge Sort porque, aunque Quick Sort tuvo mejores tiempos al comienzo, Merge Sort terminó comportándose mejor cuando aumentó el tamaño de la muestra.

En ambos casos, la estructura donde obtuvimos los mejores resultados fue ArrayList. Por esta razón, las combinaciones que escogimos para la comparación final fueron Shell Sort con ArrayList y Merge Sort con ArrayList.

Preguntas de análisis parte 2

1. **¿Cuál de los dos algoritmos seleccionados como “mejores” muestra un comportamiento más eficiente a medida que crece el tamaño de la muestra?**

Para responder, agregue líneas de tendencia en la gráfica de la pestaña 09-Bests, identifique cuál algoritmo crece más lentamente y relacione ese patrón con su complejidad teórica. Use esta comparación para justificar cuál es más escalable.

RTA:

Al comparar directamente Shell Sort y Merge Sort, vimos que Shell Sort tiene mejores tiempos en las muestras pequeñas y medianas, pero Merge Sort termina mostrando un mejor comportamiento cuando la cantidad de datos es mayor.

Con 1.000 datos, por ejemplo, Shell Sort tarda 11,01 ms y Merge Sort 17,24 ms. Con 2.000 datos la diferencia es de 23,44 ms frente a 64,71 ms y con 3.000 datos de 57,33 ms frente a 71,36 ms. Hasta ese punto Shell Sort mantiene la ventaja.

Con 5.000 datos los tiempos ya son muy parecidos: 92,05 ms para Shell Sort y 96,77 ms para Merge Sort. Después se presenta el cambio. Con 8.000 datos Merge Sort tarda 172,50 ms frente a 207,59 ms de Shell Sort y, con 10.000 datos, registra 200,77 ms frente a 272,93 ms.

Por lo tanto, aunque Shell Sort fue más rápido al inicio, Merge Sort crece más lentamente en las muestras grandes. Esto tiene relación con su complejidad $O(n \log n)$, que le permite mantener un crecimiento más controlado cuando aumenta el número de elementos.

Por esa razón consideramos que Merge Sort es el más escalable de los dos, ya que en nuestros resultados comienza a tomar ventaja precisamente cuando trabajamos con las muestras de mayor tamaño.

2. **¿Qué estructura de datos resultó más favorable para cada algoritmo seleccionado como mejor?**

Analice si la ventaja se mantiene constante en todos los tamaños o solo en ciertos rangos.

RTA:

ArrayList fue la estructura que presentó los mejores resultados. En Shell Sort, con 50 datos obtuvimos 0,39 ms en ArrayList y 1,58 ms en SingleLinkedList. Con 1.000 datos la diferencia aumentó a 11,01 ms frente a 1.176,08 ms y con 5.000 datos a 92,05 ms frente a 41.175,57 ms.

Con Merge Sort ocurrió algo parecido. Con 50 datos los tiempos fueron 0,52 ms en ArrayList y 0,87 ms en SingleLinkedList. Con 1.000 datos fueron 17,24 ms frente a 171,03 ms y, con 5.000 datos, 96,77 ms frente a 4.228,30 ms.

Por lo tanto, la ventaja de ArrayList se mantiene en todos los tamaños que pudimos comparar, pero no es una diferencia constante. En las muestras pequeñas los resultados están más cercanos y, a medida que aumenta la cantidad de datos, la diferencia se vuelve mucho mayor.

3. **¿Qué ventajas prácticas podría tener implementar el algoritmo ganador en aplicaciones reales donde se manejan grandes volúmenes de datos?**

RTA:

Una de las principales ventajas de utilizar Merge Sort sería poder manejar una cantidad cada vez mayor de datos sin que el tiempo de ejecución aumente demasiado rápido.

Otra ventaja es que su comportamiento resulta más predecible para cantidades grandes de datos. En nuestras pruebas vimos que otros algoritmos aumentaron mucho más rápido, mientras que Merge Sort mantuvo un crecimiento más estable. Por eso sería una opción más adecuada cuando se espera manejar volúmenes grandes de información.

4. **Si se presentara una lista parcialmente ordenada, ¿cambiaría su elección de algoritmo? Explique por qué, con base en el comportamiento observado.**

RTA:

Sí, porque si la lista ya está parcialmente ordenada, Insertion Sort puede aprovechar ese orden previo y necesitar menos movimientos para organizar los elementos. Por eso podría ser una mejor opción en ese caso, especialmente si la lista no es muy grande.